

0925 Answer | 把“可以做”写成可兑现的许可

原题 13 道 · 前置的线程交错 · 能力账本

先说清一个信号量代表多少个空位、多少份成品或哪一道先后门，再写 P/V。下面各段代码里的 P 是等待、V 是释放；读懂解法尚非独立限时验证。

考场入口：每题先辨识哪种门

题	最容易漏的约束	最短检查
01	奇/偶各有自己的可用数量	消费者绝不在拿锁后等待同类产品
02	10座、1台取号机、1位柜员	被叫到者离座，服务前要与柜员会合
03—05	门与容量、批次、两只邮箱	P 的顺序决定能否让退出/生产者推进
06—07	共享变量读写与哲学家环路	锁只覆盖相冲突的资源；最多同时吃 $\min(m, \lfloor n/2 \rfloor)$
08—10	前驱图与 P/V 原子性	每条跨进程边有一次可兑现的通知
11—13	原子交换、单格缓冲、植树流水	锁的真假方向、空满初值、铁锹何时释放

01 | 2009-45：同一个缓冲区，两类成品计数

45. (7分) 三个进程 P_1 、 P_2 、 P_3 互斥使用一个包含 N ($N > 0$) 个单元的缓冲区。 P_1 每次用 `produce()` 生成一个正整数并用 `put()` 送入缓冲区某一空单元中； P_2 每次用 `getodd()` 从该缓冲区中取出一个奇数并用 `countodd()` 统计奇数个数； P_3 每次用 `geteven()` 从该缓冲区中取出一个偶数并用 `counteven()` 统计偶数个数。请用信号量机制实现这三个进程的同步与互斥活动，并说明所定义信号量的含义（要求用伪代码描述）。

设 `empty=N`（空格），`odd=0`、`even=0`（已放入对应奇偶数的件数），`mutex=1`（缓冲区结构互斥）。生产者先生成 v ，`P(empty)`；`P(mutex)`；`put(v)`；`V(mutex)`；再按 v 的奇偶 `V(odd)` 或 `V(even)`。 P_2 循环 `P(odd)`；`P(mutex)`；`v=getodd()`；`V(mutex)`；`V(empty)`；`countodd(v)`； P_3 将 `odd/getodd/countodd` 换成偶数版本。`getodd/geteven` 必须从缓冲区选出该类产品，不能简单拿队首后再判奇偶。

为何要先等 `odd` 再锁缓冲区

若先 `P(mutex)` 而缓冲区里暂时无奇数， P_2 会握锁阻塞；生产者无法进入 `put` 投递奇数。先拿存在性许可，再独占实际移除动作，才能同时守住数量与互斥。

02 | 2011-45：排号不是服务完成

45. (8分) 某银行提供 1 个服务窗口和 10 个供顾客等待的座位。顾客到达银行时，若有空座位，则到取号机上领取一个号，等待叫号。取号机每次仅允许一位顾客使用。当营业员空闲时，通过叫号选取一位顾客，并为其服务。顾客和营业员的活动过程描述如下：

```
cobegin
{
    process 顾客 i
    {
        从取号机获取一个号码；
        等待叫号；
        获取服务；
    }
    process 营业员
```

```

{
    while (TRUE)
    {
        叫号；
        为客户服务；
    }
}
}coend
```

请添加必要的信号量和 `P`、`V`（或 `wait()`、`signal()`）操作，实现上述过程中的互斥与同步。要求写出完整的过程，说明信号量的含义并赋初值。

定义 `seat=10`、`machine=1`、`queueMutex=1`、`waiting=0`、`arrived=0`，并为每张唯一号码 k 提供初值 0 的 `called[k]`、`done[k]`。共享 FIFO 队列存号码。

```

顾客 i:
P(seat); P(machine); k=取唯一号码(); V(machine)
P(queueMutex); 入队(k); V(queueMutex); V(waiting)
P(called[k]); V(arrived); P(done[k]); 离开

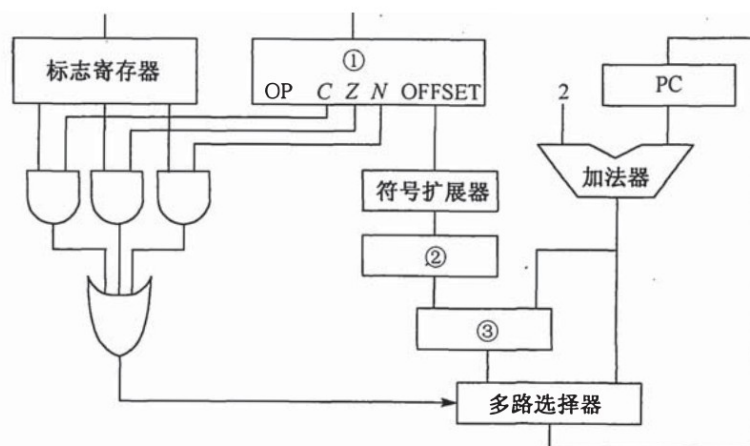
柜员 (循环):
P(waiting); P(queueMutex); k=出队(); V(queueMutex)
V(seat); 叫号(k); V(called[k]); P(arrived)
服务顾客 k; V(done[k])

```

顾客被叫号便离开等候座位；柜员等待这个顾客走到窗口才服务。machine 只限制取号机，不能被柜员握着等待顾客；done[k] 防止 A 顾客误收 B 顾客的完成通知。题目未要求先到先服务以外的插队规则，这里用队列落实取号顺序。

03 | 2013-45：先占容量，再穿过单人门

45.（7 分）某博物馆最多可容纳 500 人同时参观，有一个出入口，该出入口一次仅允许一个人通过。参观者的活动描述如下：



```

cobegin
    参观者进程 i:
    {
        ...
        进门;
        ...
        参观;
        ...
        出门;
        ...
    }
coend

```

请添加必要的信号量和 P、V（或 wait()、signal()）操作，以实现上述过程中的互斥与同步。要求写出完整的过程，说明信号量的含义并赋初值。

令 space=500 表示剩余可容纳人数，gate=1 表示出入口同一时刻只供一人穿过：

参观者 i:

P(space); P(gate); 进门; V(gate)

参观

P(gate); 出门; V(gate); V(space)

离馆完成才还一个容量名额。若把 P(gate) 放在 P(space) 前，当馆满时某人握着门等空位，馆内的人也无法出门，立刻构成死锁；这里的顺序就是本题要挖出的隐含信息。

04 | 2014-47：先让指定消费者取完连续10件

47. 系统中有多个生产者进程和多个消费者进程，共享一个能存放 1000 件产品的环形缓冲区（初始为空）。当缓冲区未空时，生产者进程可以放入其生产的一件产品，否则等待；当缓冲区未空时，消费者进程可以从缓冲区取走一件产品，否则等待。要求一个消费者进程从缓冲区连续取出 10 件产品后，其他消费者进程才可以取产品。请使用信号量 P, V（或 wait(), signal()）操作实现进程间的互斥与同步，要求写出完整的过程，并说明所用信号量的含义和初值。

2014年计算机408统考真题 第 7 页，共 7 页

设 empty=1000、full=0、bufMutex=1；另设 others=0，是其它消费者的开闸许可。多个生产者都循环生产；P(empty); P(bufMutex); 放1件; V(bufMutex); V(full)。指定消费者连续执行10次 P(full); P(bufMutex); 取1件; V(bufMutex); V(empty)，然后 V(others)。其它消费者在自己的消费循环之前先

$P(\text{others}); V(\text{others})$ ，之后每次执行同样的一件消费操作。

开闸前，其它消费者不得抢走产品；开闸后， $P(\text{others}); V(\text{others})$ 让所有消费者都能继续。指定消费者等待产品时不握 bufMutex ，生产者能继续填充。若把开闸放在指定消费者第10次 $P(\text{full})$ 之前，就不能保证“连续取出10件”。

05 | 2015-45：两只邮箱各自有空位与邮件数

45. (9分) 有 A、B 两人通过信箱进行辩论，每个人都从自己的信箱中取得对方的问题。将答案和向对方提出的新问题组成一个邮件放入对方的邮箱中。假设 A 的信箱最多放 M 个邮件，B 的信箱最多放 N 个邮件。初始时 A 的信箱中有 x 个邮件 ($0 < x < M$)，B 的信箱中有 y 个 ($0 < y < N$)。辩论者每取出一个邮件，邮件数减 1。A 和 B 两人的操作过程描述如下：

CoBegin

<pre>A{ while(TRUE){ 从 A 的信箱中取出一个邮件; 回答问题并提出一个新问题; 将新邮件放入 B 的信箱; } }</pre>	<pre>B{ while(TRUE){ 从 B 的信箱中取出一个邮件; 回答问题并提出一个新问题; 将新邮件放入 A 的信箱; } }</pre>
--	--

CoEnd

当信箱不为空时，辩论者才能从信箱中取邮件，否则需要等待。当信箱不满时，辩论者才能将新邮件放入信箱，否则需要等待。请添加必要的信号量和 P、V（或 wait、signal）操作，以实现上述过程的同步。要求写出完整过程，并说明信号量的含义和初值。

分别设 A 的 $\text{fullA}=x$, $\text{emptyA}=M-x$, $\text{mutexA}=1$, B 的 $\text{fullB}=y$, $\text{emptyB}=N-y$, $\text{mutexB}=1$ 。A 循环：

$P(\text{fullA}); P(\text{mutexA});$ 取A邮件; $V(\text{mutexA}); V(\text{emptyA});$ 回答并提出新问题; $P(\text{emptyB});$

$P(\text{mutexB});$ 放入B邮箱; $V(\text{mutexB}); V(\text{fullB});$ B 对称地 $P(\text{fullB})$ 取信，处理后 $P(\text{emptyA})$ 投递。

初始 $0 < x < M$, $0 < y < N$ ，所以双方有可读信且不满。关键是先取本箱来信、释放本箱空位，再可能等待对方箱空位；不能握本箱的互斥锁去等另一箱，或将 A 的容量许可拿来约束 B。

计数与物理操作各管什么

$\text{fullA}/\text{emptyA}$ 保证不会从空箱取或向满箱放； mutexA 只保护 A 邮箱内部的一次读写。B 有独立一组，双方能同时读写不同邮箱。

06 | 2017-46：只保护真正冲突的共享变量

46. (8分) 某进程中有3个并发执行的线程 thread1、thread2 和 thread3，其伪代码如下所示。

<pre>//复数的结构类型定义 typedef struct { float a; float b; } cnum; cnum x, y, z; // 全局变量 //计算两个复数之和 cnum add(cnum p, cnum q) { cnum s; s.a=p.a+q.a; s.b=p.b+q.b; return s; }</pre>	<pre>thread1 { cnum w; w=add(x, y); ... } thread2 { cnum w; w=add(y, z); ... }</pre>	<pre>thread3 { cnum w; w.a=1; w.b=1; z=add(z, w); y=add(y, w); ... }</pre>
---	---	--

请添加必要的信号量和 P、V (或 wait()、signal()) 操作，要求确保线程互斥访问临界资源，并且最大限度地并发执行。

x, y, z 是线程间共享的复数对象；每个线程的局部 w 与 add 的局部 s 独立。令 $my=1, mz=1$ 分别保护对全局 y, z 的访问； x 只读，无须锁。线程1 $P(my); w=\text{add}(x, y); V(my)$ ；线程2 $P(my); P(mz); w=\text{add}(y, z); V(mz); V(my)$ ；线程3 先本地构造 $w=(1, 1)$ ，随后 $P(mz); z=\text{add}(z, w); V(mz); P(my); y=\text{add}(y, w); V(my)$ 。

线程2同时读取 y, z ，故要在读两者时保留相应锁；线程3的两次独立修改分段锁定，能让线程1读 y 与线程3更新 z 并发。所有需要双锁的地方都按 $y \rightarrow z$ 取得，避免循环等待。假如额外要求“ z, y 两次更新整体不可见地完成”，题目未给这样的事务约束，才需要扩大锁范围。

07 | 2019-43：碗不消除环形拿筷子死锁

43. (8分) 有 n ($n \geq 3$) 位哲学家围坐在一张圆桌边，每位哲学家交替地就餐和思考。在圆桌中心有 m ($m \geq 1$) 个碗，每两位哲学家之间有一根筷子。每位哲学家必须取到一个碗和两侧的筷子后，才能就餐，进餐完毕，将碗和筷子放回原位，并继续思考。为使尽可能多的哲学家同时就餐，且防止出现死锁现象，请使用信号量的 P、V 操作 [wait()、signal() 操作] 描述上述过程中的互斥与同步，并说明所用信号量及初值的含义。

设 $\text{bowls}=m$ 、每根筷子 $\text{stick}[i]=1$ 、 $\text{admit}=n-1$ 。哲学家 i 循环：思考； $P(\text{admit}); P(\text{bowls}); P(\text{stick}[i]); P(\text{stick}[(i+1)\%n]);$ 吃饭； $V(\text{stick}[(i+1)\%n]); V(\text{stick}[i]); V(\text{bowls}); V(\text{admit})$ 。

最多 $n-1$ 人进入拿筷子阶段，至少留一个缺口，使环上的“每人持一根又等下一根”无法覆盖整圈；只限制碗数为 m ，若 $m \geq n$ 仍可能死锁。最多同时吃饭 $\min(m, \lfloor n/2 \rfloor)$ ：不相邻的哲学家才可同时拿到两边筷子； $n-1$ 的门槛不会削低这个最大值（ $n \geq 3$ ）。

08 | 2020-45：两处“都完成”各用一个计数门

- 45.（7分）现有5个操作A、B、C、D和E，操作C必须在A和B完成后执行，操作E必须在C和D完成后执行，请使用信号量的wait()、signal()操作（P、V操作）描述上述操作之间的同步关系，并说明所用信号量及其初值。

令 $ab=0$ ， $cd=0$ 。A后V(ab)，B后V(ab)；C之前连续P(ab)；P(ab)，然后执行C并V(cd)；D后V(cd)；E之前连续P(cd)；P(cd)再执行E。

每个前驱只交一次令牌，C/E各要收齐两枚，故先后关系是A,B→C与C,D→E，A、B、D都可并发。P(ab)一次只表示“至少有一个前驱完成”，不能当成“两个都完成”。两个计数信号量比为每条边各设一个更简洁；这题不要求互斥操作本身。

09 | 2021-45：关中断范围内不能等待别人唤醒

- 45.（7分）下表给出了整型信号量S的wait()和signal()操作的功能描述，以及采用开/关中断指令实现信号量操作互斥的两种方法。

功能描述	方法1	方法2
<pre> Semaphore S; wait(S){ while(S<=0); S=S-1; } signal(S){ S=S+1; } </pre>	<pre> Semaphore S; wait(S){ 关中断; while(S<=0); S=S-1; 开中断; } signal(S){ 关中断; S=S+1; 开中断; } </pre>	<pre> Semaphore S; wait(S){ 关中断; while(S<=0); 开中断; 关中断; } S=S-1; 开中断; signal(S){ 关中断; S=S+1; 开中断; } </pre>

请回答下列问题。

- 1) 为什么在wait()和signal()操作中对信号量S的访问必须互斥执行？
- 2) 分别说明方法1和方法2是否正确。若不正确，请说明理由。
- 3) 用户程序能否使用开/关中断指令实现临界区互斥？为什么？

(1) 互斥原因。while($S \leq 0$) 的检查与 $S = S - 1$ 必须作为一个不可分的获得许可动作；两个线程都看见 $S = 1$ 后各减1，会让同一许可被消费两次。signal 的加1也必须与 wait 对 S 的读改写保持一致。

(2) 两种错误。方法1关中断后在 while($S \leq 0$) 忙等：单处理器上释放者无法获得 CPU 执行 signal，会卡死。方法2在确认 $S > 0$ 后开中断、又关中断才减1：两个等待者可能都通过检查，随后各减1，仍竞争。安全的阻塞式信号量需在受保护的原子操作中登记等待/休眠，唤醒时维护队列与计数；单纯移动开关中断位置不能修好两者。

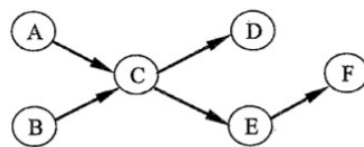
(3) 用户态。用户程序不能自行执行特权的开/关中断指令，否则可任意屏蔽系统中断；互斥要通过系统提供的同步原语。

考场反证各举一个交错

方法1取 $S = 0$ ，等待者关中断后死循环，释放者无机会运行。方法2取 $S = 1$ ，T1和T2先后都在开中断前看到1，再各减一次得到-1。

10 | 2022-46：同一线程内的先后不用重复发信号

46. (8 分) 某进程的两个线程 T1 和 T2 并发执行 A、B、C、D、E 和 F 共 6 个操作，其中 T1 执行 A、E 和 F，T2 执行 B、C 和 D。题 46 图表示上述 6 个操作的执行顺序所必须满足的约束：C 在 A 和 B 完成后执行，D 和 E 在 C 完成后执行，F 在 E 完成后执



图给 $A \rightarrow B \rightarrow C \rightarrow D, E \rightarrow F$ ；T1 负责 A、E、F，T2 负责 B、C、D。设 $aDone = 0, cDone = 0$ ：

```
T1: A; V(aDone); P(cDone); E; F
T2: B; P(aDone); C; V(cDone); D
```

$B \rightarrow C, C \rightarrow D, E \rightarrow F$ 本来就在各自线程程序顺序内；跨线程只有 $A \rightarrow C, C \rightarrow E$ 两道门，需要两个初值0的信号量。T1 发出 A 已完成后才等 C；T2 完成 C 后通知，不形成循环等待。

11 | 2023-45：原子 swap 与三个赋值不同

45. (7 分) 现要求学生使用 swap 指令和布尔型变量 lock 实现临界区互斥。lock 为线程间共享的变量。lock 的值为 TRUE 时线程不能进入临界区，为 FALSE 时线程能够进入临界区。某同学编写的实现临界区互斥的伪代码如下图 (a) 所示。

请回答下列问题。

- (1) 图 (a) 的伪代码中哪些语句存在错误？将其改为正确的语句（不增加语句条数）。
- (2) 图 (b) 给出了交换两个变量值的函数 newSwap() 的代码，是否可以用函数调用语句“newSwap(&key, &lock)”代替指令“swap key, lock”以实现临界区互斥？为什么？

某同学编写的伪代码	newSwap() 的代码
<pre>bool lock = FALSE; // 共享变量 bool key = TRUE; if (key == TRUE) 进入区 swap key, lock; // 交换 key 和 lock 的值 临界区; lock = TRUE; 退出区</pre>	<pre>void newSwap(bool *a, bool *b) { bool temp = *a; *a = *b; *b = temp; }</pre>
(a)	(b)

(1) 修正。共享 lock=FALSE 表示可进入；每个线程令局部 key=TRUE，必须 while(key==TRUE) swap key, lock;，直到从锁交换出 FALSE 才进入临界区；离开时应 lock=FALSE。图中的 if(key==TRUE) 只试一次就可能带着 TRUE 闯入，lock=TRUE 则离开后仍锁住。自旋期间 key 的 TRUE 交换回 lock，使锁保持占用。

(2) 函数替代？newSwap 用临时变量分三条赋值完成交换，两个线程可在中间交错：都读到原来的 FALSE，就都以为获得锁，或覆盖彼此写入。除非函数本身由硬件原子交换指令或等效原子机制实现，否则普通函数调用不能替代 swap key, lock。

检验锁方向的两个极端

初始 lock=FALSE，第一位应交换得到 key=FALSE 才进；若有人在内，lock=TRUE，后来者交换后仍 key=TRUE，必须反复尝试。离开必须让 lock 回到 FALSE。

12 | 2024-46：一格缓冲读写与原地修改是两种约束

46. (8 分) 计算机系统进程之间往往需要相互协作以完成一个任务。在某网络系统中，缓冲区 B 用于存放一个数据分组，对 B 的操作有 C1、C2 和 C3。C1 将一个数据分组写入 B 中，C2 从 B 中读出一个数据分组，C3 对 B 中的数据分组进行修改。要求 B 为空时才能执行 C1，B 非空时才能执行 C2 和 C3。

请回答下列问题。

- (1) 假设进程 P1 和 P2 均需要执行 C1，实现 C1 的代码是否为临界区？为什么？(2 分)
- (2) 假设 B 初始为空，进程 P1 执行 C1 一次，进程 P2 执行 C2 一次。请定义尽可能少的信号量，并用 wait()、signal() 操作描述进程 P1 和 P2 之间的同步或互斥关系，说明所用信号量的作用及其初值。(3 分)
- (3) 假设 B 初始不为空，进程 P1 和 P2 各执行 C3 一次。请定义尽可能少的信号量，并用 wait()、signal() 操作描述进程 P1 和 P2 之间的同步或互斥关系，说明所用信号量的作用及其初值。(3 分)

(1) 若 P1/P2 同时对同一缓冲 B 执行 C1，写入会互相覆盖或产生不确定结果，C1 的代码段访问共享资源 B，属于临界区。

(2) 先写后读。B 初始空，P1 的 C1 一次、P2 的 C2 一次；仅需 full=0：P1: C1; V(full); P2: P(full); C2。只有一写一读、各一次，写完后才让读发生，本题不必额外加互斥锁和 empty 信号量。

(3) 两个原地修改。B 初始非空，P1/P2 各执行一次 C3：mutex=1，两进程分别 P(mutex); C3; V(mutex)。无需等“非空”(题设已给)，但两次读改写不能交错。先后同步门 full 与资源互斥门 mutex 解决的是不同问题。

13 | 2025-45：坑容量、铁锹、树苗、水桶分开计数

45. (本题 8 分)

三个人一起植树，甲挖坑，乙放树苗入坑并填土，丙负责为新种树苗浇水。步骤依次为：挖树坑，放树苗，填土和浇水。现在有铁锹和水桶各一个，铁锹用于挖树坑，填土。水桶用于浇水。当树坑数量小于 3 时，甲才可以挖树坑。设初始坑=0，铁锹水桶均可用，定义尽可能少的信号量，用 wait() 和 signal()。操作描述植树过程中三人的同步互斥关系，并说明所用信号量的作用及其初值。

设 freePit=3 (最多还可挖几个未填的坑)、dug=0 (可放苗的坑)、planted=0 (待浇的树)、shovel=1、bucket=1。甲 P(freePit); P(shovel); 挖坑; V(shovel); V(dug); 乙 P(dug); 放苗; P(shovel); 填土; V(shovel); V(freePit); V(planted); 丙 P(planted); P(bucket); 浇水; V(bucket)，三人均按各自阶段循环。

甲挖前占一个坑名额，乙填土之后才归还：坑数量不超过3；铁锹在挖与填两个动作间可轮换，不能由甲从挖到填始终握着。水桶只在浇水时持有；乙完成种植并填土才放出 planted，保证丙不会浇尚未栽好的树。

为什么归还坑名额不必等浇水

容量条件限制的是尚未填好的坑，而浇水发生在填土之后。让甲等到丙浇完才可挖新坑会平白减少并发，且不对应题面的“坑数小于3”。

复做顺序

先看代码，为01、03、04、07、13写出“计数/初值/谁 V 后谁能 P”；再给02、06、11各构造一次会出错的交错。最终才把空位、产品、互斥和先后放进代码。能理解这种推导还需要再做独立、限时的正确性检查。